

B3 CS Infras Réseau Sécurisées > Mémos > Mémo Scapy

Mémo Scapy



Scapy permet principalement deux choses :

- crafter et envoyer des trames arbitraires
- analyser le trafic reçu
- et donc potentiellement réagir à une trame spécifique reçue

Scapy est une lib très réputée car elle donne vraiment une liberté TOTALE quand à ce qu'on met dans le paquet. Et ce, avec une syntaxe particulière, mais rapidement intuitive.

I. Crafter et envoyer des trames

Concept

Le délire de Scapy, en terme d'utilisation/syntaxe :

- on crée des **layers**
- par exemple `Ether` (pour Ethernet), par exemple `IP`, etc.
- on spécifie les **headers** de chaque *layer*
- pour le *layer* `IP` par exemple, on va spécifier au moins une adresse IP en `src` et une adresse IP en `dst`
- on emboîte les *layers* les uns dans les autres
- je crée un *layer* `DNS`, je le mets dans un *layer* `UDP`, que je mets dans un *layer* `IP`, que je mets dans un *layer* `Ether`
- une requête DNS, dans un datagramme UDP, dans un paquet IP, dans une trame Ethernet, dans un câble
- le réso koa
- notre belle trame tout belle toute mimi craftée avec nos ptites mimines, PAF, on l'envoie sur le réseau

Ca c'est une utilisation normale. Scapy = total freedom. Si tu veux mettre un Paquet, dans un Paquet, dans un Paquet, dans un Paquet. Tu peux. Ca sert à rien. Mais tu peux. Tout faire = même les trucs jamais faits sont possibles avec. Think about it.

Exemples

P'tite feuille de code toute simple (normalement ?) pour réaliser un ping de la passerelle :

```
# on importe la lib scapy
from scapy.all import *

# on craft un ping : c'est de l'ICMP
# type 8 pour ICMP echo request (le ping)
ping = ICMP(type=8)

# on craft un paquet : IP src et IP dst
# 192.168.1.29 : l'IP de mon PC chez moi
# 192.168.1.1 : l'IP de ma box chez moi, ma passerelle
packet = IP(src="192.168.1.29", dst="192.168.1.1")

# on craft une trame : MAC src et MAC dst
# d4:6d:6d:00:15:3b : la MAC de mon PC chez moi
# 78:94:b4:de:fd:c4 : la MAC de ma box chez moi, ma passerelle
frame = Ether(src="d4:6d:6d:00:15:3b", dst="78:94:b4:de:fd:c4")

# on emboîte le tout avec le caractère /
final_frame = frame/packet/ping

# srp() c'est pour send & receive
# fonction à utiliser quand on envoie un truc et qu'on attend une réponse
# ici, on envoie un ping et on attend un pong
answers, unanswered_packets = srp(final_frame, timeout=10)

# on a récupéré les pongs dans answers
# et les pings qui n'ont jamais eu de réponses sont dans unanswered_packets
print(f"Pong reçu : {answers[0]}")
```

→ **Fais-le !** J'l'attends pas dans le compte-rendu mais fais-le. Pour t'assurer que Scapy est bien setup, que tu vois bien dans Wireshark, etc, amuse-toi avec.

Un deuxième pour un ping vers 1.1.1.1 :

```
# on importe la lib scapy
from scapy.all import *

# le petit ping
ping = ICMP(type=8)

# on met 1.1.1.1 en dst pour le paquet
packet = IP(src="192.168.1.29", dst="1.1.1.1")

# MAC src et dst bouge pas : la trame va de mon PC à la passerelle
frame = Ether(src="d4:6d:6d:00:15:3b", dst="78:94:b4:de:fd:c4")
```

```
# craft trame finale
final_frame = frame/packet/ping

# send !
answers, unanswered_packets = srp(final_frame, timeout=10)

# print response !
print(f"Pong reçu : {answers[0]}")
```

→ Ok bon c'est pas super chiant de devoir préciser MAC src, MAC dst et IP src alors qu'on le fait jamais avec juste la commande `ping` ?

BAH SI en fait, surtout que c'est une fonction native et légitime de votre OS : craft des paquets normaux quoi. Tant qu'on reste dans une utilisation légit, on aimerait bien qu'il écrive les trucs évidents à notre place.

Simple : on met rien dans les champs, et si on gère pas la trame à la main, on utilise `sr()` au lieu de `srp()`.

Il existe plusieurs méthodes pour envoyer de trames avec Scapy, pas que `sr()` et `srp()`. Par exemple, il existe par exemple `send()` qui envoie sans attendre de réponse. Il existe aussi `sendp()`. Idem : on utilise `send()` si on gère pas la trame (les adresses MAC), et `sendp()` si on la gère.

```
from scapy.all import *

ping = ICMP()
packet = IP(dst="1.1.1.1")

final_frame = packet/ping

ans, unans = sr(final_frame, timeout=10)

print(f"Pong reçu : {answers[0]}")
```

→ Oh et on peut one-line aussi !

```
ans, unans = sr(IP(dst="1.1.1.1")/ICMP(), timeout=10)
print(ans)
```

→ Ca fait la même chose, mais c'est nettement plus court que le premier exemple !

N'ayez donc pas peur de la syntaxe un peu exotique de scapy. Tu mets des trucs dans des machines avec le caractère `/` et yolo.

II. Recevoir et analyser des trames

Pour analyser les trames que la machine reçoit, on utilise la fonction `sniff()`.

Elle va nous permettre de voir toutes les trames passer (qu'elle envoie ou reçoit notre machine), et d'effectuer (ou pas) une action sur chacune des trames.

On va définir une première fonction `fonction1()` : elle décrit quoi faire sur chacun des paquets. Elle recevra donc en argument une trame Scapy à chaque fois qu'une trame est reçue.

On appelle la fonction `sniff()` avec l'argument `prn=fonction1` pour indiquer à Scapy : sniff le réseau, et dès qu'une trame est reçue, tu l'envoies en argument à `fonction1`.

Bon. + de code. - de mots.

```
from scapy.all import sniff

def print_it_please(packet):
    print(f"Un petit pong qui revient de 1.1.1.1 : {packet}")

sniff(filter="icmp and src host 1.1.1.1", prn=print_it_please, count=1)
```

Le programme attend de capturer un seul paquet qui correspond au filtre (avec le `count=1`).

Une fois capturé, la fonction `print_it_please` est appelée, et Scapy lui envoie le paquet reçu en argument.

La fonction `print_it_please` s'exécute et affiche le paquet.

Le format du filtre est standard, **c'est la syntaxe BPF**, comme dans Wireshark.

On peut évidemment accéder aux différents champs de la trame reçue, un exemple :

```
from scapy.all import sniff

def print_it_please(packet):
    packet_source_ip = packet['IP'].src
    pong = packet['ICMP']
    print(f"Un petit pong qui revient de {packet_source_ip} : {pong}")

sniff(filter="icmp and src host 1.1.1.1", prn=print_it_please, count=1)
```

III. La doc

La doc de Scapy c'est une folie. Internet regorge d'exemples de bouts de code Scapy qui marchent bien.

La doc officielle devrait vous paraître plus digeste après cette petite intro, et un peu de manip de votre côté. **Cette section est chouette par exemple** : des petits one-liner qui font tout et n'importe quoi (genre y'a nativement une fonction pour MITM avec ARP).